Many to Many - Intro

We saw how to work with One-to-Many relationships in the data.

But how do you handle **Many-to-Many** relationships?

Let's explore them. 🚀

Starting from One-to-Many

Let's start with the familiar and simpler option of **One-to-Many**.

We have one table with teams and one with heroes, and for each **one** team, we can have **many** heroes.

As each team could have multiple heroes, we wouldn't be able to put the Hero IDs in columns for all of them in the `team` table.

But as each hero can belong **only to one** team, we have a **single column** in the heroes table to point to the specific team (to a specific row in the `team` table).

The `team` table looks like this:

id	name	headquarters
1	Preventers	Sharp Tower
2	Z-Force	Sister Margaret's Bar

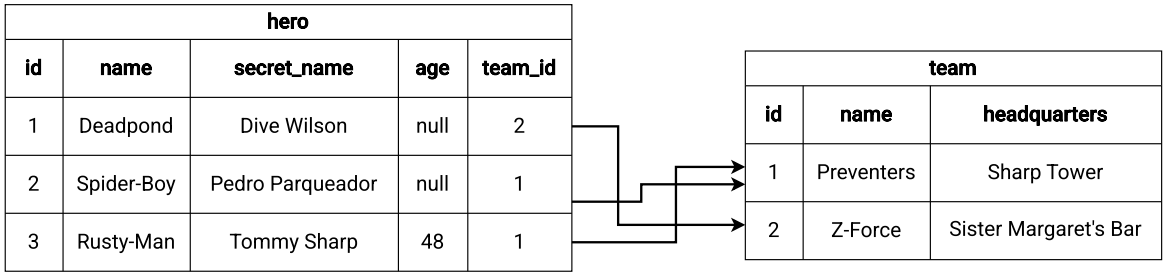
> **Tip**
>
> Notice that it doesn't have any foreign key to other tables.

And the `hero` table looks like this:

id	name	secret_name	age	team_id
1	Deadpond	Dive Wilson	null	2
2	Spider-Boy	Pedro Parqueador	null	1
3	Rusty-Man	Tommy Sharp	48	1

We have a column in the `hero` table for the `team_id` that points to the ID of a specific team in the `team` table.

This is how we connect each `hero` with a `team`:



hero				
id	name	secret_name	age	team_id
1	Deadpond	Dive Wilson	null	2
2	Spider-Boy	Pedro Parqueador	null	1
3	Rusty-Man	Tommy Sharp	48	1

team		
id	name	headquarters
1	Preventers	Sharp Tower
2	Z-Force	Sister Margaret's Bar

Notice that each hero can only have **one** connection. But each team can receive **many** connections. In particular, the team **Preventers** has two heroes.

Introduce Many-to-Many

But let's say that as **Deadpond** is a great character, they recruit him to the new **Preventers** team, but he's still part of the **Z-Force** team too.

So, now, we need to be able to have a hero that is connected to **many** teams. And then, each team, should still be able to receive **many** heroes. So we need a **Many-to-Many** relationship.

A naive approach that wouldn't work very well is to add more columns to the `hero` table. Imagine we add two extra columns. Now we could connect a single `hero` to 3 teams in total, but not more. So we haven't really solved the problem of supporting **many** teams, only a very limited fixed number of teams.

We can do better! 🤓

Link Table

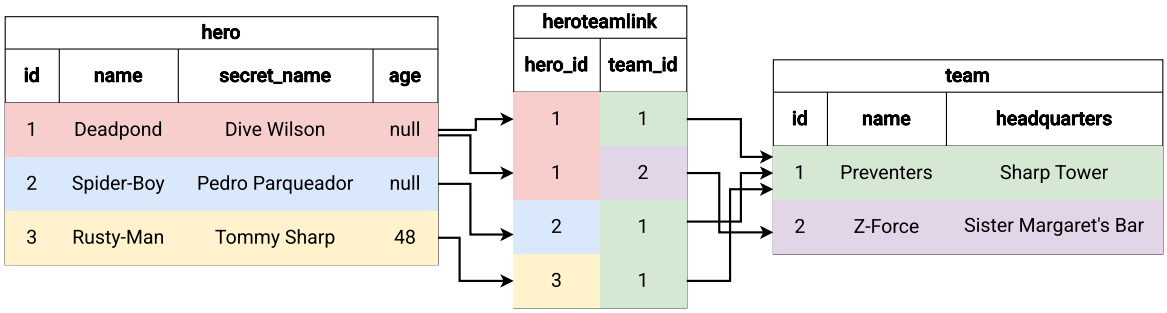
We can create another table that would represent the link between the `hero` and `team` tables.

All this table contains is two columns, `hero_id` and `team_id`.

Both columns are **foreign keys** pointing to the ID of a specific row in the `hero` and `team` tables.

As this will represent the **hero-team-link**, let's call the table `heroteamlink`.

It would look like this:



hero			
id	name	secret_name	age
1	Deadpond	Dive Wilson	null
2	Spider-Boy	Pedro Parqueador	null
3	Rusty-Man	Tommy Sharp	48

heroteamlink	
hero_id	team_id
1	1
1	2
2	1
3	1

team		
id	name	headquarters
1	Preventers	Sharp Tower
2	Z-Force	Sister Margaret's Bar

Notice that now the table `hero` **doesn't have a `team_id`** column anymore, it is replaced by this link table.

And the `team` table, just as before, doesn't have any foreign key either.

Specifically, the new link table `heroteamlink` would be:

hero_id	team_id

1	1
1	2
2	1
3	1

Info

Other names used for this **link table** are:

- association table
- secondary table
- junction table
- intermediate table
- join table
- through table
- relationship table
- connection table

I'm using the term "link table" because it's short, doesn't collide with other terms already used (e.g. "relationship"), it's easy to remember how to write it, etc.

Link Primary Key

Cool, we have a link table with **just two columns**. But remember that SQL databases require each row to have a **primary key** that **uniquely identifies** the row in that table?

Now, what is the **primary key** in this table?

How to we identify each unique row?

Should we add another column just to be the **primary key** of this link table? Nope! We don't have to do that. 👌

Both columns are the primary key of each row in this table (and each row just has those two columns). ✨

A primary key is a way to **uniquely identify** a particular row in a **single table**. But it doesn't have to be a single column.

A primary key can be a group of the columns in a table, which combined are unique in this table.

Check the table above again, see that **each row has a unique combination** of `hero_id` and `team_id`?

We cannot have duplicated primary keys, which means that we cannot have duplicated links between `hero` and `team`, exactly what we want!

For example, the database will now prevent an error like this, with a duplicated row:

hero_id	team_id
1	1

1	2
2	1
3	1
3 🚨	1 🚨

It wouldn't make sense to have a hero be part of the **same team twice**, right?

Now, just by using the two columns as the primary keys of this table, SQL will take care of **preventing us from duplicating** a link between `hero` and `team`. ✅

Recap

An intro with a recap! That's weird... but anyway. 🤷

Now you have the theory about the **many-to-many** relationships, and how to solve them with tables in SQL. 🤓

Now let's check how to write the SQL and the code to work with them. 🚀